

Del CSV histórico a una probabilidad de victoria: cómo se combinan los modelos

Tres modelos estadísticos independientes — uno por partido, uno por set, uno por punto — encadenados dentro de un motor de simulación de Cadenas de Markov. Este documento traza el flujo completo, con los números reales de cada etapa y qué partes están confirmadas frente a las que siguen abiertas.

• 142 tests automatizados en verde • 725 partidos / 5073 sets, 2016–2026 • Última auditoría de precisión: julio 2026

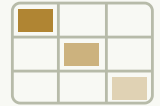
AUC PARTIDO (MATCH) 0.75 Elo con margen, test 2025/26. Antes: 0.53 real (0.707 reportado, con fuga de datos).	AUC SET (SET, CV HONESTO) 0.63 ± 0.08 Validación rolling-origin. El 0.70 aislado de 2025 no está confirmado como estructural.	COBERTURA DE TESTS 142 Suite completa en verde tras cada integración, incluye el adapter del modelo v2.	DATOS HISTÓRICOS 725 / 5073 Partidos y sets recorridos cronológicamente para construir el Elo.
--	--	--	---

EL FLUJO, DE PRINCIPIO A FIN

Cinco etapas, un único sentido: de los CSV a la clasificación final

Cada etapa consume la salida de la anterior. El orden importa: los ratings Elo, por ejemplo, sólo son válidos si se recorren los partidos en orden cronológico.

DB/*.csv → src/data/data_pipeline.py



Datos: el histórico de la SuperLiga, set a set

`sets_partidos.csv` es la fuente de verdad: 725 partidos / 5073 sets con marcador punto por punto agregado, 10 temporadas (2016/17–2025/26). Se complementa con historiales por equipo, enfrentamientos directos y stats de jugadores.

Los nombres de equipo llegan sucios entre fuentes ("*Diatec Trentino*", "*MonzaMonza*"...); `team_mapper.normalize_team_name()` los unifica antes de usarlos como clave en cualquier etapa posterior.

src/data/rolling_features.py



Features sin fuga: recorriendo el calendario en orden

La primera versión de las features (`match_features.csv`, 87 columnas) mezclaba estadísticas de la temporada *completa* en partidos de esa misma temporada — un partido de octubre "conocía" el resultado de abril. Auditado y sustituido por un constructor que recorre `sets_partidos.csv` cronológicamente y sólo usa información estrictamente anterior.

Produce, entre otras: **Elo con margen de victoria**, forma reciente (EWMA), enfrentamientos directos con decaimiento temporal y ratios de set/punto en ventana móvil.

```
E_h = 1 / (1 + 10^((R_a - R_h - ADV) / 400))
R_h' = R_h + K · margin_mult · (resultado - E_h)
margin_mult = 1 + 0.15 · (|Δsets| - 1) → 3-0 pesa 1.30×, 3-2 pesa 1.00×
K = 28    ADV = 60    R_base = 1500    regresión entre temporadas = 25%
```

`get_historical_team_elo()``get_historical_team_strengths()`

Tres modelos, tres niveles de granularidad

src/data/rolling_features.py · src/models/set_predictor_v2.py ·
src/models/point_probability.py

El sistema no usa un único clasificador: predice a tres escalas distintas del mismo partido, y cada escala alimenta a la siguiente dentro del motor de simulación.

A · NIVEL PARTIDO

MATCH — Elo con margen

Probabilidad de que el local gane el partido. Sin entrenamiento: es determinista, sale directo del rating. Sustituyó a un XGBoost de 87 features cuyo "AUC 0.707" resultó ser la fuga de la etapa 02.



B · NIVEL SET

SET — Regresión logística

Probabilidad de ganar el set actual, con 21 features de contexto (fuerza, forma, marcador de sets previos, momentum). Regularizada (C=0.5) y ponderada por recencia (half-life 2 temporadas).



[⚠ ver honestidad científica](#)

C · NIVEL PUNTO

POINT — Regresión logística

Probabilidad de ganar el punto siguiente, en función de la diferencia de fuerza y de quién saca. Ajustada por *sideout rate* (61.9%, el equipo que recibe domina el punto). Es la señal que realmente mueve la Cadena de Markov.

$$p_{\text{recibe}} = p_{\text{base}} \cdot S0 / (p_{\text{base}} \cdot S0 + (1 - p_{\text{base}}) \cdot (1 - S0))$$

S0 (sideout) = 0.62

+ [Detalle técnico: PlayerStatsGenerator](#)

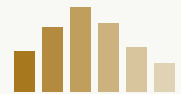


Motor de simulación: Cadena de Markov punto a punto

Cada punto es un estado que se decide con la probabilidad del modelo POINT, ajustada por **momentum** (racha intra-set +0.015 por punto, tope 4; momentum global entre sets) y recortada por un **clamp** — un rango duro que evita probabilidades extremas.

El clamp puede ser fijo `[0.20, 0.80]` o adaptativo, recentrado cada set en la probabilidad que da el modelo SET de la etapa anterior. El set termina a 25 puntos (15 en el quinto) con diferencia de 2; el partido, al mejor de cinco sets.

⚠ el clamp adaptativo está diagnosticado con ruido — ver honestidad científica



Agregación: de un set a una temporada

Partido suelto: se repite la simulación N veces (hasta 5000) con distinta semilla y se agrega en una distribución de resultados — $P(\text{gana local})$, y el reparto entre 3-0, 3-1, 3-2, etc.

Temporada completa: se recorre el calendario round-robin jornada a jornada; el Elo runtime se actualiza con cada resultado simulado (usando la misma fórmula de la etapa 02), así que la fuerza de los equipos evoluciona dentro de la propia simulación. Al final: la clasificación.

LA PREGUNTA CENTRAL

Cómo se combinan los tres modelos dentro de un único partido

No son tres predicciones independientes que se promedian: forman una cadena, donde la salida de un modelo se convierte en el input del siguiente.

MODELO A · MATCH

Elo con margen → p_elo

Probabilidad de que el local gane el partido, calculada desde el rating histórico de ambos equipos.

calibra fuerzas (damping 0.5)

FUERZAS DEL PARTIDO

strength_local, strength_visitante

Las fuerzas base de la etapa 02 se recalibran con p_elo antes de simular un solo punto.

alimenta el modelo de punto

MODELO C · POINT

Regresión logística → p_saque / p_recibe

Probabilidad de ganar el punto siguiente, ajustada por sideout rate (62%).

entra a la Cadena de Markov

MOTOR MARKOV + MODELO B · SET

Punto a punto, con momentum y clamp

Cada punto se decide con $p_{\text{punto}} \pm \text{momentum}$, recortado a un rango. Ese rango se recentra cada set con la probabilidad del modelo SET — el eslabón con el caveat abierto (ver más abajo).

mejor de 5 sets

RESULTADO

Marcador del partido

Un partido simulado. Repetido N veces (Monte Carlo) o encadenado jornada a jornada (temporada) da la distribución o la clasificación final.

PARA LA DEFENSA

Honestidad científica: qué está confirmado y qué sigue abierto

El hallazgo metodológico más importante del proyecto no es una mejora de precisión: es haber detectado que la precisión reportada originalmente era, en parte, un espejismo.

✓ CONFIRMADO Y CORREGIDO

Fuga de datos en MATCH

El AUC 0.707 original usaba features de temporada completa filtradas al pasado. Medido honesto: 0.53. Corregido con Elo limpio: 0.75.

Selección de modelo sobre ruido

El "campeón" se elegía sobre 81 partidos de validación (error estándar ± 0.06 en AUC). Sustituido por validación rolling-origin multi-temporada.

Bug de sembrado del Elo runtime

Un import faltante hacía que el simulador cayera en silencio a un Elo plano. Detectado al diagnosticar el clamp, corregido y verificado con asserts de comportamiento, no sólo ausencia de warnings.

Constantes Elo desincronizadas

El motor de runtime usaba $K=32/ADV=65$ mientras el histórico usaba $K=28/ADV=60$ — train/serve skew clásico. Centralizadas en un único origen.

⚠ CON CAVEAT, PENDIENTE

El "0.70" del SET v2 es 2025-específico

Validación año a año: 2018–2024 se mueven en 0.57–0.64: la CV honesta da 0.63 ± 0.08 , prácticamente empatada con el modelo legacy (0.62 ± 0.03). Pendiente re-validar con la temporada 2026/27.

El clamp adaptativo aporta ruido, no señal

Sobre 132 pares de equipos, la correlación entre el clamp y el Elo limpio es $\rho \approx 0$. Añade +22% de varianza a la posición final y $\times 60$ el coste de cómputo. Plan de corrección documentado, sin ejecutar.

Sin backtest completo del motor de momentum

Los parámetros de momentum y el damping de calibración nunca se contrastaron contra una temporada real completa a nivel de Brier score.

Reproducir el pipeline

```
python -m src.data.data_pipeline      # sanity-check de los CSV
python -m src.models.train_improved    # Elo v2 + SET v2, snapshot de precisión
python -m src.models.measure_precision # métricas honestas, antes/después
python -m pytest -q                    # 142 tests
python -m src.api.main                 # API en :8000
```

[memoria/mejora_precision_2026-07.md](#) — bitácora completa

[PLAN_MEJORA_CLAMP.md](#) — diagnóstico y plan del clamp [COMPARACION_ANTES_DESPUES.md](#) — cifras exactas